# Analiza Amortizată - CheatSheet

## Obiective (1)

- Fie o secvență de operații care acționează asupra unui structuri de date.
- Vrem să obținem o estimare precisă a timpului de execuție pentru întreaga secvență.

## Obiective (2)

- Timpul de execuție al fiecărei operații depinde de starea internă a structurii de date.
- Astfel, același tip de operație se poate desfășura uneori lent, alteori foarte rapid.
- Ne propunem să demonstrăm că numărul de operații rapide este suficient de mare pentru ca costul mediu (*costul amortizat*) al unui tip de operație, raportat la costul întregii secvențe, să fie relativ mic.

## Observații

- Analiza se va concentra asupra operațiilor care modifică structura de date.
- Analiza amortizată diferă de analiza medie probabilistică (*average-case analysis*) prin faptul că nu face presupuneri referitoare la distribuția datelor de intrare.
- Rezultatele obținute sunt exacte, aceleași indiferent de natura datelor de intrare.

## Notații

- $n$ - Numărul de operații din secvență.
- $c$ - Costul real al unei operații.
- $\hat{c}$ - Costul amortizat al unei operații.
- $T(n)$ - Costul real al întregii secvențe.

## Metoda 1 - Analiză Agregată

- Calculăm timpul **maxim** necesar pentru execuția întregii secvențe de operații ($T(n)$).
- Asociem un cost amortizat unic pentru toate operațiile din secvență: $\hat{c} = \frac{T(n)}{n}$ (**valoarea medie**).

## Metoda 2 - Metoda Creditelor (1)

- Alegem un cost amortizat pentru fiecare tip de operație care să **supraestimeze** costul real al operațiilor rapide și să **subestimeze** costul operațiilor lente.
- **Obiectiv**: $\sum_{i=1}^{n} \hat{c}_i \geqslant \sum_{i=1}^{n} c_i$.

## Metoda Creditelor (2) - Observații

- Dacă $\hat{c}_i > c_i$, diferența o vom 'salva' într-un cont virtual.
- "Creditul" acumulat dupa mai multe astfel de operații îl vom folosi pentru a plăti pentru o viitoare operație costisitoare, pentru care costul amortizat ales este mult mai mic decât costul real.
- Alegerea este corectă dacă creditul acumulat este întotdeauna **pozitiv**, indiferent de ordinea operațiilor din secvență.

## Metoda Creditelor (3) - Notații

- $credits_i$ - **Creditul** acumulat dupa primele $i$ operații.
- $credits_i = credits_{i-1} + \hat{c}_i - c_i$.
- $\forall i > 0.\ credits_i \geq 0, \hat{c}_i \geq 0$.

## Metoda Potențialului (1)

- Similar cu metoda creditelor, folosim **diferența dintre costul amortizat și cel real** pentru a înmagazina, virtual, "energie potențială", pe care o vom folosi atunci când întâlnim o operație costisitoare.
- Vom defini o funcție de potențial care să descrie 'energia' disponibilă în structura de date, după primele operații realizate.

## Metoda Potențialului (2) - Notații

- $D_0$ - Structura de date inițială.
- $D_i$ - **Structura de date** care rezultă după aplicarea operației $i$ asupra structurii de date $D_{i-1}$
- $\phi : D \to \mathbb{R}$ - **Funcția de potențial**.
- $\hat{c}_i = c_i + \phi(D_i) - \phi(D_{i-1})$.
- $\forall i > 0.\ \phi(D_i) \geq \phi(D_0), \hat{c}_i \geq 0$.

## Metoda Potențialului (3) - Observații

- Pentru a fi utilă, funcția de potențial trebuie să descrie o proprietate influențată direct de **numărul** de operații efectuate.
- Altfel, dacă nu există o variație de potențial între momentele aplicării unei operații, costul amortizat va fi egal cu cel real.

## Observații Generale

- Ultimele două metode pot asocia costuri amortizate diferite pentru tipuri diferite de operații, spre deosebire de analiza agregată.
- Un cost amortizat este **valid** atât timp cât suma tuturor costurilor amortizate este cel puțin la fel de mare ca suma costurilor reale, pentru **orice secvență** posibilă de operații.
- Prin metodele menționate ne propunem să identificăm un cost amortizat **minim**.

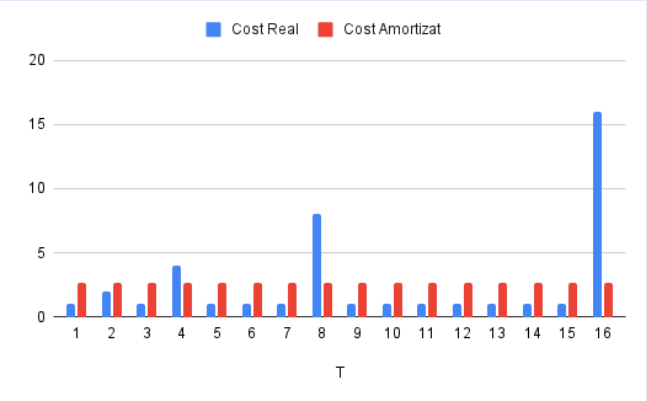
# Aplicații (1)

## Exercițiul 1 (1)

Fie o secvență de operații aplicate pe o structură de date generică, unde costul real al operației cu indicele $i$ din secvență are asociat costul:

$$c_i = \begin{cases} i, & \text{dacă i este putere a lui 2} \\ 1, & \text{altfel} \end{cases}$$

## Exercițiul 1 (2) - Analiză Agregată




- Aplicăm formula: $\hat{c}_i = \frac{T(n)}{n} \in O(1)$

## Exercițiul 1 (3) - Observații

- **Insight**: Dacă **numărul** operațiilor care se execută în timp constant este direct proporțional cu **costul** operației lente care le succede, atunci putem asocia un cost amortizat constant pentru orice operație din secvență.
- Nu întotdeauna costul amortizat pentru o operație, obținut conform metodelor menționate, va fi constant.

## Exercițiul 2 (1)

Fie o structură de tip vector redimensionabil care implementează operațiile:

- `add(elem)` - Adaugă un element la finalul secvenței curente.
- `at(index)` - Returnează, în timp constant, elementul de pe poziția 'index'.

Parametrii structurii de date:

- `Cap` - Capacitatea (spațiul total alocat).
- `Size` - Numărul efectiv de elemente din secvență.

Pentru operația `add()`, dacă numărul de elemente din secvență depășește capacitatea vectorului, se alocă un vector nou cu o capacitate dublă în care se mută elementele din vectorul precedent.

## Exercițiul 2 (2) - Metoda Creditelor

Pasul 1: Explicităm costul real pentru operația cu cost variabil `add(elem)`.

$$c_{add} = \begin{cases} c_{copy} \cdot (Cap + 1), & \text{dacă Size} = \text{Cap} \\ c_{copy}, & \text{altfel} \end{cases}$$

## Exercițiul 2 (3) - Metoda Creditelor

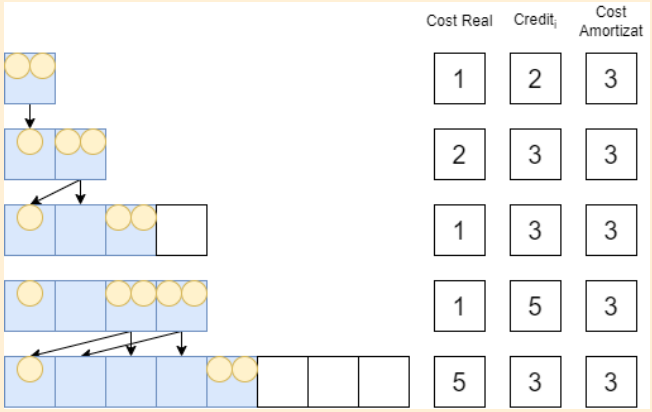
- Costul operației `at` nu este influențat de poziția în secvență și nu afectează costul operației `add`.
- Prin urmare, nu îi putem asocia decât un cost amortizat egal cu cel real (constant).

## Exercițiul 2 (4) - Metoda Creditelor

- **Intuiție**: Între două realocări de memorie, avem $\frac{Cap}{2}$ operații care se execută rapid.
- În acest interval, trebuie să acumulăm suficient credit pentru a "plăti" pentru viitoarea realocare.

## Exercițiul 2 (5) - Metoda Creditelor

- Pasul 2: Optăm pentru un cost amortizat care să 'aloce' fiecărui element nou adăugat suficiente unități de credit pentru a suporta operația propriu-zisă de adăugare, respectiv viitoarea mutare a cel puțin două elemente ($\hat{c}_{add} \geq 3 * c_{copy}$).




- Pasul 3: Analizăm pentru primele valori ale lui $n$ și deducem o formulă generală pentru creditul acumulat.
  - $P(i) : credit_i = c_{copy} * (2 * (Size_i - \frac{Cap_i}{2}) + 1)$.
- Pasul 3: Validăm, prin inducție, că vom acumula întotdeauna suficient credit pentru a plăti pentru o operație scumpă.
  - $P(i) \implies P(i+1)$ . $\forall i > 1$

## Exercițiul 2 (6) - Metoda Potențialului

- Alternativ, putem proceda în oglindă alegând o funcție de potențial cu o valoare direct proporțională cu numărul de operații rapide, ulterioare unei operații lente.
- $\phi(D_i) = c_{copy} * k * (Size_i - \frac{Cap_i}{2})$.
- Analizăm cele două cazuri posibile pentru a justifica că putem aloca un cost amortizat constant ($\hat{c}_{add} = 3 * c_{copy}$) dacă alegem $k = 2$.

# Aplicații (2)

## Exercițiul 1 - Follow-ups (1)

- Costul amortizat calculat poate să nu fie constant, scopul său este doar să ne ajute să obținem o estimare mai precisă a costului întregii secvențe:

$$C1_i = \begin{cases} i^2, & \text{dacă i este putere a lui 2} \\ i, & \text{altfel} \end{cases}$$

- $\hat{C1_i} \in O(n)$

## Exercițiul 1 - Follow-ups (2)

- Dacă numărul de operații rapide executate între două operații lente nu este direct proporțional cu costul următoarei operații lente, nu vom avea suficiente oportunități pentru a acumula credit/energie potențială.
- Prin urmare, nu vom putea obține un cost amortizat mai bun decât cel necesar în cazul cel mai defavorabil.

$$C2_i = \begin{cases} i, & \text{dacă i este multiplu al lui K} \\ 1, & \text{altfel} \end{cases}$$

- $\hat{C2_i} \in O(n)$

## Exercițiul 2 - Follow-up (1)

- Ne propunem să implementăm și o operație de tip `pop`, care elimină ultimul element de la sfârșitul vectorului și reduce `Cap` vectorului dacă `Size` scade prea mult ($Size = \frac{Cap}{t}$).
- Pentru $t = 2$, costul amortizat pentru `add` și `pop` va fi egal cu cel mai defavorabil (`Size`). (De ce?)
- Pentru $t > 2$, putem exprima funcția de potențial diferit, în funcție de relația dintre numărul de elemente și capacitate.
- Astfel, dacă $Size_i \geq \frac{Cap_i}{2}$, cu fiecare operație de adăugare vom înmagazina energie pentru a putea plăti pentru o posibilă dublare a capacității vectorului în viitor.
- Altfel, dacă $Size_i < \frac{Cap_i}{2}$, cu fiecare operație de ștergere vom înmagazina energie pentru a putea plăti pentru o posibilă înjumătățire a capacității vectorului.

$$\phi(D_i) = \begin{cases} k_1 * (Size_i - \frac{Cap_i}{2}), & Size_i \geq \frac{Cap_i}{2} \\ k_2 * (\frac{Cap_i}{2} - Size_i), & \text{altfel} \end{cases}$$